

Operating Systems Project

Phase 4 Report

Group Members:

Yixuan Ma ym3116

Uditi Sharma us2133

1 Architecture and Design

The system runs three types of threads:

- **Main thread** — accepts TCP connections, spawns one client thread per connection.
- **Client threads** — receive commands from connected clients and enqueue them as Task objects. They do not execute commands.
- **Scheduler thread** — dequeues tasks, applies the scheduling algorithm, executes programs via `fork()/execvp()`, and streams output back to each client's socket.

The scheduler owns a singly-linked task queue protected by a mutex and condition variable. All scheduling state (queue, running task, timeline) lives in `scheduler.c`; `server.c` interacts with it only through the public API in `scheduler.h`.

Scheduling algorithm: SJRF + Round Robin

- Shell commands (`burst_time = -1`) bypass the queue and run immediately without preemption.
- Program tasks (`./demo N`) are queued and selected by shortest remaining time (SJRF).
- Each task gets a quantum: 3 seconds on its first round, 7 seconds on subsequent rounds.
- If a new task with strictly shorter remaining time arrives while a program is running, the running task is preempted via `SIGSTOP` and the new task is promoted.
- A task cannot be selected twice in a row if other tasks exist (anti-consecutive rule).
- Equal remaining times are broken by arrival order (FCFS).

2 Implementation Highlights

Task Structure (`scheduler.h`)

Each command submitted by a client becomes a Task on the scheduler's queue. The key fields that drive scheduling decisions are:

```
typedef struct Task {
    int      task_id;
    int      client_fd;
    int      burst_time;      // original N; -1 for shell commands
    int      remaining_time;  // seconds still to run
    int      round;          // rounds completed so far
    pid_t    child_pid;      // 0 if not yet started
    int      out_pipe[2];     // child stdout → scheduler
    time_t   arrival_time;    // for FCFS tie-breaking
    size_t   total_bytes_sent;
```

```

    struct Task *next;
} Task;

```

remaining_time decreases each quantum and is the primary sort key for SJRF. round distinguishes the first quantum (3s) from subsequent ones (7s). child_pid is non-zero between the first fork() and final waitpid(), allowing the scheduler to send SIGSTOP/SIGCONT to freeze and resume the process across quanta. out_pipe stays open for the lifetime of the child so output can be read across multiple scheduling rounds.

SJRF Selection with Anti-Consecutive Rule (scheduler.c)

```

for (Task *t = g_queue_head; t; t = t->next) {
    if (count > 1 && t->task_id == g_last_run_id) continue;
    if (!best || t->remaining_time < best->remaining_time ||
        (t->remaining_time == best->remaining_time &&
         t->arrival_time < best->arrival_time))
        best = t;
}

```

The queue is scanned linearly each scheduling round. The task with the shortest remaining_time wins (SJRF). If two tasks have equal remaining time, the one that arrived earlier wins (FCFS tiebreak via arrival_time). The anti-consecutive check (g_last_run_id) prevents the same task from being selected back-to-back when other tasks are waiting, which ensures fairness and prevents starvation of waiting tasks.

Preemption Check (scheduler.c)

```

static int check_preemption(Task *running) {
    pthread_mutex_lock(&g_queue_mutex);
    for (Task *t = g_queue_head; t; t = t->next) {
        if (t->remaining_time < running->remaining_time) {
            pthread_mutex_unlock(&g_queue_mutex);
            return 1;
        }
    }
    pthread_mutex_unlock(&g_queue_mutex);
    return 0;
}

```

This function is called after every output line forwarded from the running child. It scans the queue while holding the mutex and returns 1 if any waiting task has a strictly shorter remaining time than the current task. When it returns 1, `forward_output_for()` breaks out of its loop early, and the caller immediately sends `SIGSTOP` to freeze the child — achieving preemptive SJRF without a dedicated timer thread.

Streaming Output with `select()` (`scheduler.c`)

```
struct timeval tv = { .tv_sec = 1, .tv_usec = 100000 }; // 1.1s
timeout
int ready = select(fd + 1, &rfd, NULL, NULL, &tv);
if (ready <= 0) break; // child stopped or finished

ssize_t n = read(fd, &c, 1);
if (n <= 0) return -1; // EOF: child exited naturally

if (c == '\n') {
    uint32_t pkt_len = htonl((uint32_t)buf_pos);
    send_all_fd(t->client_fd, &pkt_len, sizeof(pkt_len));
    send_all_fd(t->client_fd, line_buf, buf_pos);
    t->total_bytes_sent += buf_pos;
    elapsed++;
    if (check_preemption(t)) break;
}
```

`forward_output_for()` reads the child's pipe one byte at a time, assembling complete lines. Each completed line is immediately forwarded to the client as a length-prefixed packet so the client sees output in real time. The 1.1-second `select()` timeout detects when the child has been `SIGSTOP`'d (no more output) without blocking indefinitely. After each line, `check_preemption()` is called — this is the point where the scheduler decides whether to yield the CPU to a higher-priority task mid-quantum.

`SIGSTOP` / `SIGCONT` Lifecycle (`scheduler.c`)

```
// First execution: fork the child
pid = fork();
// ...
t->child_pid = pid;
// Quantum expires or preemption: freeze the child
kill(t->child_pid, SIGSTOP);
```

```
// Next scheduling round: thaw and continue
kill(t->child_pid, SIGCONT);
// Task complete: reap
waitpid(t->child_pid, &status, 0);
```

On the first scheduling round the child process is forked and begins executing. When the quantum expires (or a preemption is triggered), SIGSTOP suspends the child at its current position — its pipe, file descriptors, and memory remain intact. On the next round, SIGCONT resumes it exactly where it left off, continuing to write to the same pipe that the scheduler reads. This lets a single child process span multiple scheduling rounds without restarting.

3 Execution Instructions

Build: `make clean && make`

(This produces: `myshell_server`, `myshell_client`, `myshell`, and `demo`.)

Start the server: `./myshell_server 9090`

Connect a client: `./myshell_client 127.0.0.1 9090`

4 Testing

All three required test scenarios were run on the remote Linux server with the server and clients compiled via `make clean && make`. Full output for each scenario is demonstrated in the demo video.

Test Scenario 1 — Three Concurrent Clients, Single Shell Commands

Three clients connect simultaneously, each sending one shell command (`pwd`, `ls`, `echo hello`). Shell commands bypass the queue and run immediately with burst time `-1`. The server log shows `created → started → ended (-1)` for each client with no preemption and no timeline summary.

Result: PASS

Test Scenario 2 — One Shell Command + Two Demo Programs

One client sends `pwd` (shell command) and two others send `./demo 12`. The shell command runs instantly. The two demo programs are scheduled via SJRF + RR, interleaving with `waiting` and `running` labels visible in the server log. A colored timeline summary is printed when both finish.

Result: PASS

Test Scenario 3 — Three Demo Programs with Different N Values

Three clients send `./demo 12`, `./demo 10`, and `./demo 8` in quick succession. SJRF preemption is clearly visible — shorter jobs interrupt longer ones mid-quantum. The shortest job (demo 8) finishes first, the longest (demo 12) finishes last. The timeline summary shows the full interleaved execution order.

Result: PASS

Summary

#	Test	Result
1	3 concurrent clients, single shell commands	PASS
2	1 shell command + 2 demo programs	PASS
3	3 demo programs with different N values	PASS

5 Challenges

1. Designing Preemption Without a Dedicated Timer Thread

The most difficult design decision was how to preempt a running program task at the right moment without adding a separate timer thread. The approach taken was to check for preemption after each line of output forwarded from the child process (i.e., once per second, since `demo` prints one line per second). `check_preemption()` scans the queue under the mutex and

returns 1 if any waiting task has a strictly shorter remaining time. `forward_output_for()` then breaks its loop early, and the caller sends `SIGSTOP` to freeze the child. This eliminated the need for a timer thread entirely and kept the scheduler single-threaded.

2. Preserving Child State Across Scheduling Rounds via `SIGSTOP/SIGCONT`

A subtlety of the `SIGSTOP/SIGCONT` approach is that the child's pipe file descriptors must remain open across multiple scheduling rounds. If the write end of the pipe were closed after the first quantum, the child would get a `SIGPIPE` on its next `printf`. The fix was to keep `out_pipe[0]` (read end) open in the scheduler and `out_pipe[1]` (write end) open in the child until the child exits naturally or is reaped. This allows the scheduler to resume reading from the same pipe on the next `SIGCONT` round without restarting the child.

3. The Anti-Consecutive Rule and Equal Remaining Times

Implementing the requirement that the same task cannot be selected twice in a row (when other tasks exist) required tracking `g_last_run_id` across scheduling rounds. The edge case is when the anti-consecutive skip would leave no eligible tasks — in that case the rule must be relaxed and the only remaining task must be allowed to run again. Handling this correctly without introducing starvation required careful logic in `select_next_task()`.

4. Detecting Child Exit vs. `SIGSTOP` in `forward_output_for()`

When a child is stopped via `SIGSTOP`, it stops writing to the pipe — but the pipe is not closed, so a blocking `read()` would hang indefinitely. The solution was to use `select()` with a 1.1-second timeout on the pipe read end. If `select()` times out, the child is either stopped or has finished; if `read()` returns 0 (EOF), the child exited naturally. This distinguished the two cases without requiring `waitpid(WNOHANG)` polling.

5. Thread-Safe Logging Across Three Thread Types

With a main thread, a scheduler thread, and multiple client threads all potentially calling `printf()` at the same time, log lines would interleave without protection. A single `g_print_mutex` is declared in `scheduler.c` and declared `extern` in `scheduler.h` so `server.c` can share it. Every log statement in both files acquires this mutex, ensuring each log line is written atomically.

5 Challenges

1. Designing Preemption Without a Dedicated Timer Thread

The most difficult design decision was how to preempt a running program task at the right moment without adding a separate timer thread. The approach taken was to check for preemption after each line of output forwarded from the child process (i.e., once per second, since `demo` prints one line per second). `check_preemption()` scans the queue under the mutex and returns 1 if any waiting task has a strictly shorter remaining time. `forward_output_for()` then breaks its loop early, and the caller sends `SIGSTOP` to freeze the child. This eliminated the need for a timer thread entirely and kept the scheduler single-threaded.

2. Preserving Child State Across Scheduling Rounds via `SIGSTOP/SIGCONT`

A subtlety of the `SIGSTOP/SIGCONT` approach is that the child's pipe file descriptors must remain open across multiple scheduling rounds. If the write end of the pipe were closed after the first quantum, the child would get a `SIGPIPE` on its next `printf`. The fix was to keep `out_pipe[0]` (read end) open in the scheduler and `out_pipe[1]` (write end) open in the child until the child exits naturally or is reaped. This allows the scheduler to resume reading from the same pipe on the next `SIGCONT` round without restarting the child.

3. The Anti-Consecutive Rule and Equal Remaining Times

Implementing the requirement that the same task cannot be selected twice in a row (when other tasks exist) required tracking `g_last_run_id` across scheduling rounds. The edge case is when the anti-consecutive skip would leave no eligible tasks — in that case the rule must be relaxed and the only remaining task must be allowed to run again. Handling this correctly without introducing starvation required careful logic in `select_next_task()`.

4. Detecting Child Exit vs. `SIGSTOP` in `forward_output_for()`

When a child is stopped via `SIGSTOP`, it stops writing to the pipe — but the pipe is not closed, so a blocking `read()` would hang indefinitely. The solution was to use `select()` with a 1.1-second timeout on the pipe read end. If `select()` times out, the child is either stopped or has finished; if `read()` returns 0 (EOF), the child exited naturally. This distinguished the two cases without requiring `waitpid(WNOHANG)` polling.

5. Thread-Safe Logging Across Three Thread Types

With a main thread, a scheduler thread, and multiple client threads all potentially calling `printf()` at the same time, log lines would interleave without protection. A single `g_print_mutex` is declared in `scheduler.c` and declared `extern` in `scheduler.h` so `server.c` can share it. Every log statement in both files acquires this mutex, ensuring each log line is written atomically.

6 Division of Tasks

Task	Team Member
Implementation (scheduler.c, scheduler.h, demo.c, server.c, client.c, Makefile)	Annie
Architecture, Design, and Implementation Highlights sections of report	Annie
Testing all 3 scenarios, recording video and verifying output correctness	Uditi
Testing, Challenges, Division of Tasks, and References sections of report	Uditi

7 References

1. **POSIX Signals: `SIGSTOP`, `SIGCONT`, `SIGKILL` — Linux Programmer's Manual.** Used to suspend and resume child processes between scheduling quanta. Available via `man 7 signal` on any POSIX system.
2. **`fork(2)`, `execvp(3)`, `waitpid(2)` — Linux Programmer's Manual.** Used for spawning child processes to run program tasks and reaping them on completion.
3. **`select(2)` — Linux Programmer's Manual.** Used in `forward_output_for()` to detect child `SIGSTOP` (pipe silence) versus natural exit (EOF) without blocking indefinitely.

4. **POSIX Threads (pthreads) Programming — Open Group Base Specifications Issue 7, IEEE Std 1003.1-2017.** Used for `pthread_create`, `pthread_mutex_lock`, `pthread_cond_wait`, `pthread_detach`.
<https://pubs.opengroup.org/onlinepubs/9699919799/>
5. **Beej's Guide to Network Programming — Brian "Beej" Hall.** Reference for the TCP socket API and the length-prefix framing protocol. <https://beej.us/guide/bgnet/>
6. **W. Richard Stevens, Bill Fenner, Andrew M. Rudoff — "UNIX Network Programming, Volume 1: The Sockets Networking API", 3rd Edition.** Prentice Hall, 2003. Reference for concurrent server design using threads and TCP socket programming patterns.
7. **`pipe(2)`, `dup2(2)` — Linux Programmer's Manual.** Used to redirect child `stdout/stderr` into a pipe that the scheduler reads to stream output back to the client.
8. **`htonl(3)`, `ntohl(3)` — Network byte order conversion, Linux Programmer's Manual.** Used for the 4-byte length-prefix framing protocol between server and client.